RETO TÉCNICO – HABICAPITAL 2027

Especificación de requerimientos

Sistema de transferencias y gestión de dinero entre personas

Documento:	Requerimientos funcionales, no funcionales y entregables
Versión:	0.7
Estado:	Aplicación local con Flask y JavaScript
Fecha:	Noviembre de 2026

Documento de trabajo

Índice

1. Alcance del sistema **2**

2. Requerimientos funcionales **2**

3. Propuesta de valor: Juntia **3**
3.1. Requerimientos de Juntia 3
3.2. Límites conocidos de Juntia 4

4. Requerimientos no funcionales **4**
4.1. Usability (Usabilidad) 4
4.2. Reliability (Confiabilidad) 4
4.3. Performance (Rendimiento) 5
4.4. Supportability (Mantenibilidad y soporte) 5
4.5. Restricciones adicionales (+): Seguridad 6
4.6. Restricciones de datos e implementación 6

5. Tecnologías del proyecto **7**

6. Entregables **7**

7. Referencia **8**

1. Alcance del sistema

El sistema permitirá administrar cuentas de usuarios y realizar operaciones de transferencia de dinero entre ellas mediante una aplicación web local, que se ejecuta en el equipo del usuario. Su propósito principal es permitir que los usuarios puedan crear cuentas, cargar saldo de manera simulada, transferir dinero entre cuentas internas, consultar sus saldos y visualizar el historial de movimientos realizados.

Debido a la naturaleza financiera del sistema, se establece como prioridad garantizar la integridad, consistencia y trazabilidad de las operaciones, evitando la pérdida, duplicación o alteración injustificada de dinero.

Quedan fuera de esta versión: publicación en internet, despliegue en servidor, grupos y gastos compartidos (salvo Juntia, la propuesta de valor descrita en su sección), reparto de montos, bloqueo de cuentas, monedas distintas de COP, pasarelas de pago reales e inteligencia artificial.

2. Requerimientos funcionales

Los requerimientos funcionales describen las operaciones principales que el sistema deberá ofrecer a sus usuarios.

ID	Requerimiento	Descripción
RF-01	Registro y creación de cuenta	El sistema debe registrar un usuario con nombre único y contraseña de al menos 8 caracteres, e iniciar su cuenta con saldo de 0 COP.
RF-02	Inicio y cierre de sesión	El sistema debe permitir iniciar sesión con nombre de usuario y contraseña para acceder a las funciones de la cuenta, e invalidar el acceso anterior al cerrar sesión.
RF-03	Carga de saldo	El sistema debe permitir cargar saldo en la cuenta propia mediante cargas simuladas de entre 1 y 5.000.000 COP, registrando cada carga como movimiento con contrapartida en la cuenta de fondeo del sistema.
RF-04	Transferencia de dinero	El sistema debe permitir transferir saldo desde una cuenta propia a una cuenta existente y distinta, por montos de entre 1 y 5.000.000 COP, y rechazar la transferencia sin crear movimientos si el saldo es insuficiente.
RF-05	Consulta de saldo	El sistema debe permitir consultar el saldo actual en COP de una cuenta propia.
RF-06	Historial de movimientos	El sistema debe permitir consultar los movimientos de una cuenta propia, ordenados del más reciente al más antiguo, en bloques de 20.

3. Propuesta de valor: Juntia

Juntia es un bolsillo financiero compartido en el que dos o más personas reúnen dinero para alcanzar un objetivo común, como pagar una cena, un viaje o los gastos de un apartamento compartido. Cada integrante aporta desde su cuenta personal, consulta el saldo acumulado y revisa los movimientos del bolsillo.

El valor está en que nadie puede gastar el dinero común sin acuerdo: todo retiro requiere la aprobación explícita de los integrantes activos. Así, el manejo del dinero compartido queda transparente y trazable, y reutiliza las garantías de integridad del sistema (movimientos inmutables, suma cero y atomicidad) sin agregar piezas nuevas.

Juntia es una extensión de valor añadido. No reemplaza ningún requerimiento anterior y es la única forma de grupo permitida en esta versión.

3.1 Requerimientos de Juntia

ID	Requerimiento	Descripción
RF-07	Creación de bolsillo	El sistema debe permitir crear un bolsillo con nombre de hasta 40 caracteres. El creador queda como miembro activo y el bolsillo inicia con saldo de 0 COP.
RF-08	Invitación y aceptación	Un miembro activo debe poder invitar a otro usuario de Habi. El invitado debe aceptar la invitación antes de ver el saldo o los movimientos del bolsillo.
RF-09	Aporte al bolsillo	Un miembro activo debe poder aportar desde su cuenta personal, por montos de entre 1 y 5.000.000 COP. Si su saldo es insuficiente, el sistema debe rechazar el aporte sin crear movimientos.
RF-10	Propuesta de retiro	Un miembro activo debe poder proponer un retiro del bolsillo hacia la cuenta de cualquier usuario de Habi. El monto propuesto queda reservado: el saldo disponible descuenta los retiros pendientes. Si el monto supera el saldo disponible, el sistema debe rechazar la propuesta.
RF-11	Aprobación de retiros	Un retiro debe ejecutarse solo cuando aprueban todos los miembros activos del momento en que se propuso. Un rechazo debe cancelarlo y liberar la reserva. Un retiro sin respuesta durante 7 días debe expirar y liberar la reserva.

ID	Requerimiento	Descripción
RF-12	Consulta y salida	Un miembro activo debe poder consultar el saldo, el saldo disponible, los miembros, los retiros y los movimientos del bolsillo. Un miembro debe poder salir del bolsillo solo si no tiene aportes ni votos pendientes.

3.2 Límites conocidos de Juntia

- El saldo es compartido y no tiene dueño por persona. Quien aportó no puede salir del bolsillo, y esta versión no define un mecanismo para cerrarlo.
- Los aportes y retiros se registran en el libro de movimientos como transferencias.
- No hay notificaciones: los miembros se enteran de invitaciones y retiros al consultar el bolsillo.

4. Requerimientos no funcionales

Los requerimientos no funcionales establecen las características de calidad y las restricciones que deberá cumplir el sistema. Se organizan de acuerdo con las categorías correspondientes del modelo FURPS+.

4.1 Usability (Usabilidad)

ID	Requerimiento	Descripción
RNF-U-01	Retroalimentación	El sistema debe mostrar en la página si cada operación fue exitosa o, si falló, el motivo del error.
RNF-U-02	Presentación de información	El sistema debe mostrar los montos en COP con separador de miles (por ejemplo, 50.000 COP) y las fechas en formato ISO 8601.
RNF-U-03	Validación de entradas	El sistema debe señalar los datos inválidos indicando el campo, antes de ejecutar la operación, sin alterar saldos ni movimientos.

4.2 Reliability (Confiabilidad)

ID	Requerimiento	Descripción
RNF-R-01	Persistencia	El sistema debe mantener almacenadas las operaciones con respuesta exitosa después de reiniciar el servidor local y la aplicación.

ID	Requerimiento	Descripción
RNF-R-02	Consistencia ante fallos y atomicidad	El sistema debe registrar ambos movimientos de cada transferencia o ninguno, y dejar los saldos como estaban antes si una operación falla.
RNF-R-03	Idempotencia	El sistema debe generar una clave única para cada formulario de transferencia y, al reenviar la misma clave, devolver el resultado original sin crear movimientos adicionales.
RNF-R-04	Concurrencia	El sistema debe impedir que transferencias simultáneas desde la misma cuenta superen su saldo o lo dejen negativo. Prueba: 20 transferencias simultáneas de 1.000 COP desde una cuenta con 10.000 COP deben producir 10 exitosas y saldo final de 0 COP.
RNF-R-05	Idempotencia en aportes y retiros de Juntia	Los aportes y las propuestas de retiro deben usar una clave única. Reenviar la misma clave no debe crear aportes ni retiros adicionales.

4.3 Performance (Rendimiento)

ID	Requerimiento	Descripción
RNF-P-01	Tiempo de respuesta	En el equipo de desarrollo, el sistema debe completar una transferencia o consulta de saldo en menos de 1 segundo.

4.4 Supportability (Mantenibilidad y soporte)

ID	Requerimiento	Descripción
RNF-S-01	Mantenibilidad	El sistema debe separar la lógica de transferencias de las rutas web, de las plantillas y del acceso a datos.
RNF-S-02	Capacidad de prueba	El sistema debe permitir ejecutar toda la suite automatizada con un único comando, con al menos una prueba por requerimiento funcional.

ID	Requerimiento	Descripción
RNF-S-03	Documentación	El sistema debe incluir un README con las decisiones técnicas, los mecanismos de integridad (ENT-02) y la advertencia de que el servidor de desarrollo de Flask solo se usa en local.

4.5 Restricciones adicionales (+): Seguridad

ID	Requerimiento	Descripción
RNF-SEC-01	Autenticación	El sistema debe exigir una sesión iniciada para las funciones de cuenta, guardar la sesión en una cookie firmada que expire a los 60 minutos y almacenar las contraseñas únicamente como hash.
RNF-SEC-02	Autorización	El sistema debe permitir consultar el saldo e historial, o transferir desde una cuenta, únicamente a su dueño, y rechazar cualquier otro intento.
RNF-SEC-03	Autorización en bolsillos	Solo los miembros activos de un bolsillo deben poder ver su saldo y movimientos, aportar, proponer retiros o votar. Un usuario invitado o ajeno no debe acceder a ninguno de estos datos.

4.6 Restricciones de datos e implementación

ID	Requerimiento	Descripción
RNF-D-01	Precisión monetaria	El sistema debe expresar los montos en COP como enteros, sin decimales.
RNF-D-02	Validación de montos	El sistema debe rechazar montos menores o iguales a 0, montos no enteros y montos mayores a 5.000.000 COP por operación.
RNF-D-03	Integridad referencial	El sistema debe hacer que cada movimiento referencie una cuenta existente e impedir eliminar cuentas con movimientos.
RNF-D-04	Inmutabilidad	El sistema debe impedir modificar o eliminar movimientos, y registrar cada corrección como un movimiento nuevo.

ID	Requerimiento	Descripción
RNF-D-05	Conservación del dinero	El sistema debe generar movimientos que sumen 0 COP en cada operación, y permitir saldo negativo únicamente en la cuenta de fondeo, contrapartida de las cargas.
RNF-D-06	Conservación en bolsillos	Cada operación de un bolsillo debe generar movimientos que sumen 0 COP, y el saldo de un bolsillo nunca debe ser negativo.

5. Tecnologías del proyecto

El sistema se implementará con las siguientes tecnologías. La lógica de negocio (transferencias y reglas de saldo) se mantiene independiente de estas herramientas, de acuerdo con RNF-S-01.

Capa	Tecnología	Uso
Lenguaje	Python 3.14	Lógica de negocio y rutas del servidor.
Servidor local	Flask	Rutas web en `localhost`, sin publicación en internet.
Interfaz	HTML, CSS y JavaScript con plantillas Jinja2	Páginas de acceso y de cuenta. JavaScript sin frameworks para mejoras puntuales.
Sesión	Cookie firmada de Flask	Mantiene la sesión iniciada y se invalida al cerrar sesión.
Base de datos	SQLite (módulo `sqlite3`)	Almacenamiento en un archivo local, con transacciones.
Contraseñas	`hashlib.pbkdf2_hmac`	Almacenamiento de contraseñas como hash, sin dependencias externas.
Identificadores	`uuid`	Generación de claves únicas para las transferencias.
Pruebas	pytest y cliente de pruebas de Flask	Pruebas unitarias, de concurrencia, de integración y de rutas.
Control de versiones	Git y GitHub	Historial del código y repositorio público del reto.

6. Entregables

ID	Entregable	Descripción
ENT-01	Repositorio	Repositorio público en GitHub con el código del sistema.
ENT-02	README	En primera persona, responde: decisiones clave, protección del saldo, funcionalidades excluidas, qué harías distinto, qué no sabes, supuestos, uso de IA (incluido al menos un error de la IA) y aprendizajes.
ENT-03	Video	Máximo 5 minutos, con demo funcional y decisiones clave explicadas en voz alta.
ENT-04	Correo de entrega	Enviado a `tech-team@habicapital.com` con nombre, enlace al repositorio, enlace al video, CV y enlace a portafolio o GitHub personal.

7. Referencia

HabiCapital. (2026). *Reto Técnico – Convocatoria 2027: Practicantes, Equipo de Tecnología*. Documento proporcionado para el proceso de selección.